

• AUTONOMOUS DEBUGGING, WITH RECEIPTS

Fix the bug. See the proof.

Agentic Code Fixer turns a failing snippet into a tested patch. No black box, no hand-waving — every inference, edit, and test run stays in view.

[Try the fixer](#) →

[Understand the method](#)

 LOCAL-FIRST DEMO

 YOUR CODE STAYS YOURS

AGENT TRACE / 0046

reproducible

01 Find the assumption

profile can be null

02 Contain the failure

guarded access + fallback

03 Prove the change

2 / 2 tests passing

confidence
high / 0.94

1.64s

01 / THE WORKSPACE

Put the failure on the table.

Paste a real bug or start with the fixture. The agent will narrate its work from first read to final assertion.

untitled-failure

LANGUAGE TypeScript

SOURCE INPUT3 lines

REPAIR LOOP

```
1 function getUsername(user: User) {
2   return user.profile.name.toUpperCase();
3 }
4
5
6
7
```

1 ANALYSIS

2 DIAGNOSIS

3 PATCH

4 RE-RUN

5 VALIDATION

AGENT OUTPUT

The agent is waiting for a failure

Run the fixture above to watch a transparent repair session.

☐ Simulate a validation edge case

Test contract attached

Run repair

2. TESTS (optional)

```
it("returns a user's display name", () => {
  expect(getUsername({ profile: { name: "Ada" } })).toBe("ADA");
  expect(getUsername({ profile: null })).toBe("UNKNOWN");
});
```

02 / METHOD

A loop you can audit.

The agent earns trust by narrowing uncertainty in public. Each phase leaves a useful artifact, not just a green check.

01

Analyze

Builds a small map of symbols, paths, and the exact failing surface.

02

Diagnose

Connects the observed failure to a specific assumption in your code.

03

Patch

Prefers a narrow, readable edit over a clever rewrite or dependency leap.

04

Validate

Re-runs the contract, checks the diff, and tells you when it cannot prove safety.

03 / FIELD NOTES

Built for the moment after panic.

[Read the docs](#) →

📄 QUICKSTART.MD

```
import { fixer } from '@agentic/fixer'\n\nconst result = await fixer.repair([\n  code,\n  tests,\n  language: 'typescript'\n])\n\n// result.diff, result.tests, result.reasoning
```

Q4 / API
ACCESS

Make the repair loop part of your stack.

One endpoint for
code, context, and
tests. Stream the
reasoning trace
into your own
review surface or CI
logs.

Copy API
starter

POST /v1/repair

200 OK

```
{\n  "status": "validated",\n  "confidence": 0.94,\n  "patch": {\n    "files": 1,\n    "tests_passed": 2,\n    "reasoning_url": "/traces/0040"\n  }\n}
```

Start small. Scale with evidence.

NO CREDIT CARD FOR LOCAL RUNS

Local

\$0 / month

For your next stubborn bug

- ✓ Unlimited local runs
- ✓ Reasoning trace
- ✓ Copyable patches

Choose Local →

Team

MOST USED

\$49 / month

For teams shipping weekly

- ✓ 5,000 repair runs / month
- ✓ PR and CI integrations
- ✓ Shared trace history

Choose Team →

Enterprise

Let's talk

For codebases with a perimeter

- ✓ Private deployment
- ✓ SAML and audit exports
- ✓ Dedicated model routing

Talk to the team →

Autonomous does not mean unsupervised.

Keep source inside your network, route sensitive workloads to your approved models, and give every repair a trace your reviewers can sign off on.

[Plan your perimeter →](#)



Private by default

No source retention. Bring your own model key.



Reviewable changes

Policy gates before a patch can merge.



Observable runs

Export traces to the tools your team already trusts.